

Course code and name:	B37VB-Praxis Programming
Type of assessment:	Individual <i>(delete as appropriate)</i>
Coursework Title:	Game Project
Student Name:	Aanya Rath
Student ID Number:	H00504716

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the University's Regulations on plagiarism as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature *(type your name):* AANYA RATH

Date: 31/03/2026

Copy this page and insert it into your coursework file in front of your title page. For group assessment each group member must sign a separate form and all forms must be included with the group submission.

Your work will not be marked if a signed copy of this form is not included with your submission.

Alien alert! - The game

Contents

Alien alert! - The game	2
Revision History	2
Introduction	2
How to Play	2
Logic of the game	3
Conclusion	4
Resources	5

Revision History

Date	Author Name	Version	Notes
29 th March	Aanya Rath	1	Created Document
30 th March	Aanya Rath	2	Added introduction
31 st March	Aanya Rath	3	Added how to play section
31 st March	Aanya Rath	4	Added logic of the game section
1 st April	Aanya Rath	5	Proofread report

Introduction

The main gist of the game is to shoot an alien that keeps respawning. The player character is the blue circle, and the alien is the purple circle. The bullets you shoot are the tiny red circles. If you miss an alien three times, it's game over. You can quit the game by pressing escape. If you only have one life left, and no score the alien will slow down until the score is 10. When the player hits score 10 the speed of the alien increases as it is approaching the player.

How to Play

You can move the player character by pressing 'A' and 'D' on the keyboard to move left and right. By pressing 'SPACE' you can shoot bullets at the alien. Before playing the game, there is a title screen with a logo that stays for 2 seconds, and then by pressing 'ENTER' or clicking on the screen you can start the game. If you lose, a game over screen will appear, where you can press 'ENTER' to try again. You can quit the game by pressing 'ESCAPE'.

Logic of the game

All the variables for the speed of the player, bullet and alien are all initialized first in the main function. The image for the alien logo is also loaded here as a texture. I defined variables for the four different screens in the game: the logo, the title, the gameplay and the game over screen. There is a switch case statement for when to switch over from one screen to another.

For the logo screen, a variable to count the number of frames is initialized which is translated into a delay time. In the code, if our counter reaches more than 120 frames (2 seconds) it will switch the current screen to title. To go from the title to gameplay, the player can press enter or click on the screen.

The gameplay screen has a score and lives counter. Three variables are initialized before the while loop that contains the game loop, one for the times the player missed the alien, another for the number of lives the player gets and the score counter. The lives left are calculated by subtracting the times the player missed the alien. The player is spawned at the top-center of the screen initially. The alien spawns at the bottom of the screen at a random X coordinate but a fixed Y coordinate.

In the loop, anytime the player presses A, it will add the movement speed to the X coordinate of the player, similarly if the player presses D, it will subtract the movement speed from the X coordinate of the player. Each time any changes are made to the X coordinate, the player is drawn on screen using raylib's drawcircle function, which is in a function called spawnPlayer. If the space key is pressed, it draws the bullet initially from the player's position and keeps adding the bullet movement speed to its Y coordinate value. Again, like the player, we use the drawcircle function every time any change is made to the bullet's position, so it looks like the bullet is moving forward. Now, if the bullet hits the alien, which is checked using the collision between circles function, it clears the screen and spawns another alien. To spawn the alien in a random position, a function to generate a random X-coordinate was created called spawnAlienX, which returns an int value. This is done with the help of the rand function.

For the alien to move forward, in another if statement, we check if the Y coordinate of the alien is within the screen height and we keep subtracting the alien speed from the Y coordinate. When the player loses the alien, which is when the Y coordinate of the alien is equal to 0, it adds to the player loss variable. The alien is also spawned using the draw function continuously, so that it is animated.

Once the player loss variable has reached the value 3, it breaks the game loop and moves on to the game over screen in the switch case statement. Since the game loop keeps running until the player has lost or pressed escape to quit, if the player has reached a score of 10 the speed of the alien is increased. Otherwise, if the player has a score of 0 and has two alien losses then the alien speed is reduced to make it easier for the player. The game logic is in a while loop, which will remain running until the window is closed.

The game over screen, has the option to either try again or quit the game by pressing escape.

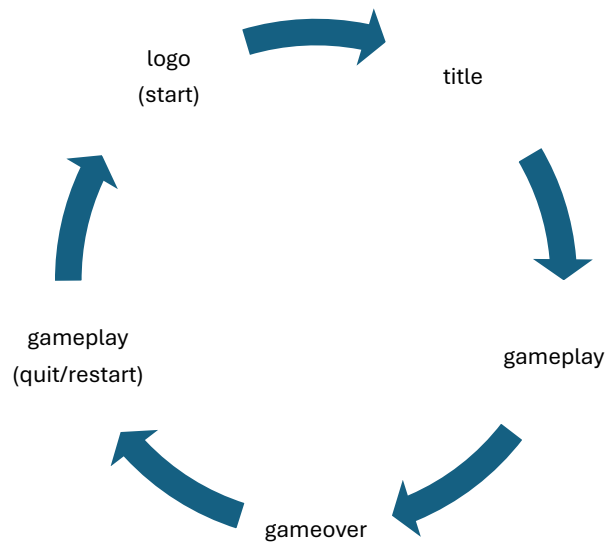


Figure 1. main game loop

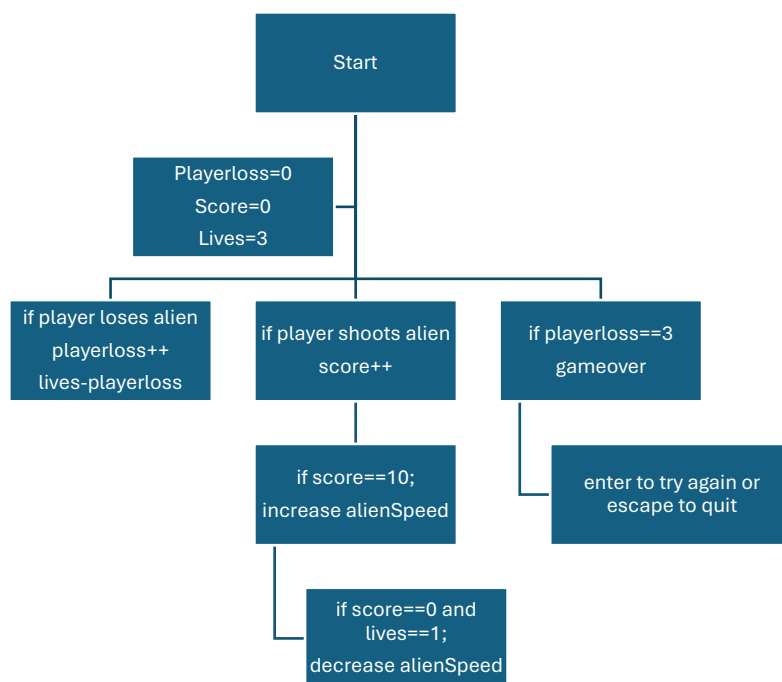


Figure 2. game logic/system

Conclusion

Overall, the game works with simple logic. For the alien, the bullet and the player, I have kept them as circles, as they are much simpler to animate and are easier to work with when it comes

to detecting collisions. I have taken help from some raylib's examples, such as the logo, the title, gameplay and ending screen switching. The player's movement was easily implemented. The challenging part was animating the bullet and alien. Sometimes the alien would disappear, and the bullet would not shoot. I had to create multiple files, if I was making any major changes to the code. Most of the problems, were fixed with a lot of debugging and playtesting.

Resources

<https://www.pinterest.com/pin/7529524368375335/> - Alien Logo image

<https://www.raylib.com/cheatsheet/cheatsheet.html>

https://www.raylib.com/examples/textures/loader.html?name=textures_image_loading

https://www.raylib.com/examples/core/loader.html?name=core_basic_screen_manager